

Program Analysis 2025-26

Lecture #04

Total Correctness and Invariants



UNIVERSITÀ
DI PISA

The HL rules

$$\frac{}{\{P\} \text{ skip } \{P\}}$$

$$\frac{}{\{P\} x := a \{ \exists x'. P[x'/x] \wedge x = a[x'/x] \}}$$

$$\frac{}{\{Q[a/x]\} x := a \{Q\}}$$

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}}$$

$$\frac{\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}}$$

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{ while } b \text{ do } c \{P \wedge \neg b\}}$$

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}}$$

Euclidean division algorithm

The Euclidean division algorithm computes the quotient and the remainder of the division of an integer x by a positive integer y .

$r := x$

$q := 0;$

while $y \leq r$ do

$r := r - y;$

$q := q + 1$

we want to prove that it is correct!

At the end given x and $y > 0$, it produces integers q and r such that

$$x = qy + r \text{ and } 0 \leq r < y$$

$$\{\text{true}\} \quad C \quad \{x = qy + r, 0 \leq r < y\}$$

Hoare's proof

{true}

$r := x$

$q := 0;$

while $y \leq r$ do

$r := r - y;$

$q := q + 1$

Hoare's proof

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\}$$

$q := 0;$

while $y \leq r$ do

$r := r - y;$

$q := q + 1$

$$\{Q[a/x]\} x := a \{Q\}$$

Hoare's proof

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$q := 0;$

$$\{x = r + qy\}$$

while $y \leq r$ do

$r := r - y;$

$q := q + 1$

$$\{Q[a/x]\} x := a \{Q\}$$

Hoare's proof

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$q := 0;$

$$\{x = r + qy\}$$

while $y \leq r$ do

$r := r - y;$

$q := q + 1$

loop invariant?

$$\{P \wedge b\} c \{P\}$$

$$\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}$$

Hoare's proof

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$q := 0;$

$$\{x = r + qy\}$$

loop invariant?

while $y \leq r$ do

$$\{x = r + qy \wedge y \leq r\}$$

$r := r - y;$

$q := q + 1$

$$\{x = r + qy\}$$

$$\{x = r + qy \wedge y > r\}$$

$$\{P \wedge b\} c \{P\}$$

$$\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}$$

Hoare's proof

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$q := 0;$

$$\{x = r + qy\}$$

loop invariant?

while $y \leq r$ do

$$\{x = r + qy \wedge y \leq r\} \Rightarrow \{x = r + qy\}$$

consequence
rule

$r := r - y;$

$q := q + 1$

$$\{x = r + qy\}$$

$$\{x = r + qy \wedge y > r\}$$

$$\{P \wedge b\} c \{P\}$$

$$\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}$$

Hoare's proof

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$q := 0;$

$$\{x = r + qy\}$$

loop invariant?

while $y \leq r$ do

consequence
rule

$$\{x = r + qy \wedge y \leq r\} \Rightarrow \{x = r + qy\} \equiv \{x = (r - y) + (q + 1)y\}$$

$r := r - y;$

$$\{x = r + (q + 1)y\}$$

$q := q + 1$

$$\{x = r + qy\}$$

$$\{x = r + qy \wedge y > r\}$$

$$\{P \wedge b\} c \{P\}$$

$$\frac{}{\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}}$$

$$\frac{}{\{Q[a/x]\} x := a \{Q\}}$$

Wait a moment...

$$\{\text{true}\} \equiv \{x = x\}$$

$$r := x$$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$$q := 0;$$

$$\{x = r + qy\}$$

while $y \leq r$ do

$$\{x = r + qy \wedge y \leq r\} \Rightarrow \{x = (r - y) + (q + 1)y\}$$

$$r := r - y;$$

$$\{x = r + (q + 1)y\}$$

$$q := q + 1$$

$$\{x = r + qy\}$$

$$\{x = r + qy \wedge y > r\}$$

$$[[c]][x \mapsto 5, y \mapsto -2] = \dots = \emptyset$$

Wait a moment...

$$\{\text{true}\} \equiv \{x = x\}$$

$r := x$

$$\{x = r\} \equiv \{x = r + 0y\}$$

$q := 0;$

$$\{x = r + qy\}$$

while $z = 0$ do

$$\{x = r + qy \wedge z = 0\} \Rightarrow \{x = (r - y) + (q + 1)y\}$$

$r := r - y;$

$$\{x = r + (q + 1)y\}$$

$q := q + 1$

$$\{x = r + qy\}$$

$$\{x = r + qy \wedge z \neq 0\}$$

$$\llbracket c \rrbracket [x \mapsto 5, y \mapsto 2, z \mapsto 0] = \dots = \emptyset$$

No guarantee of termination

$$\{x \geq 0\}$$

while $x > 0$ do

$$\{x \geq 0 \wedge x > 0\} \equiv \{x + 1 \geq 0\}$$

$x := x + 1;$

$$\{x \geq 0\}$$

$$\{x \geq 0 \wedge x \leq 0\} \equiv \{x = 0\}$$

$$\llbracket c \rrbracket [x \mapsto 5] = \dots = \emptyset$$

False positive

$\{x = 1\}$ while $x > 0$ do $x := x + 1$ $\{x = 0\}$

complete the proof below

not a possible
output!

$\{x = 1\} \Rightarrow \{?\}$

while $x > 0$ do

$\{? \wedge x > 0\}$

$x := x + 1;$

$\{?\}$

$\{? \wedge x \leq 0\} \Rightarrow \{x = 0\}$

Partial vs total correctness

when the precondition is met,
executing the command
establishes the postcondition



partial

$\{P\} c \{Q\}$

when the precondition is met,
executing the command **terminates**
and establishes the postcondition



total

total correctness = partial correctness + termination

Total correctness: the idea

Choose a an arithmetic expression on program variables, called t , such that

- t decreases at every iteration
- t is bounded below
- therefore the loop cannot execute infinitely many times

This gives termination.

$\{P\}$
while b do
 $\{P \wedge b\}$
 c
 $\{P\}$
 $\{P \wedge \neg b\}$

choose a measure
called "variant"
(e.g., an arithmetic expression
over program variables)

t

here z is a fresh variable
that keeps the value of t
before the execution of c

$t = z$

$t < z$

prove that each execution
of the body c
decreases the value of t

prove that
whenever the loop invariant holds
the value of t is bounded below

$t \geq 0$

Rule for total correctness

in red the two proof obligations for termination

Whenever the invariant and the guard hold, executing the body strictly decreases the variant

The diagram illustrates the rule for total correctness. It features a horizontal line separating the preconditions from the conclusion. Above the line, three blue callout boxes point to specific parts of the red-shaded precondition: 'variant = termination function' points to t , 'fresh variable' points to z , and 'precedence relation' points to $<$. Below the line, a blue callout box labeled 'bounded decrease' points to the expression $P \Rightarrow t \geq 0$.

$$\frac{\{P \wedge b\} \ c \ \{P\} \quad \{P \wedge b \wedge t = z\} \ c \ \{t < z\} \quad P \Rightarrow t \geq 0}{\{P\} \ \text{while } b \ \text{do } c \ \{P \wedge \neg b\}}$$

Whenever the invariant holds, the variant is bounded below.

Total correctness proof

$\{x \geq 0\}$ **take** $t \triangleq x$

while $x > 0$ **do**

$\{x \geq 0 \wedge x > 0\} \equiv \{x - 1 \geq 0\}$

$x := x - 1;$

$\{x \geq 0\}$

$\{x \geq 0 \wedge x \leq 0\} \equiv \{x = 0\}$

proof
obligations

$P \Rightarrow t \geq 0$

$x \geq 0 \Rightarrow x \geq 0$

$\{P \wedge b \wedge t = z\} \text{ c } \{t < z\}$

$\{x \geq 0 \wedge x > 0 \wedge x = z\} \Rightarrow$

$\{x = z\} \Rightarrow$

$\{x < z + 1\} \equiv$

$\{x - 1 < z\} \Rightarrow$

$x := x - 1$

$\{x < z\}$

Total correctness proof

$$\{x \geq 0 \wedge y > 0\} \equiv \{x \geq 0 \wedge y > 0 \wedge x = x + 0y\}$$

$r := x$

$$\{x \geq 0 \wedge y > 0 \wedge x = r + 0y\} \equiv \{r \geq 0 \wedge y > 0 \wedge x = r + 0y\}$$

$q := 0;$

$$\{r \geq 0 \wedge y > 0 \wedge x = r + qy\}$$

while $y \leq r$ do **take** $t \triangleq r$

$$\{r \geq y > 0 \wedge x = r + qy\} \Rightarrow \{r - y \geq 0 \wedge y > 0 \wedge x = r - y + (q + 1)y\}$$

$r := r - y;$

$$\{r \geq 0 \wedge y > 0 \wedge x = r + (q + 1)y\}$$

$q := q + 1$

$$\{r \geq 0 \wedge y > 0 \wedge x = r + qy\}$$

$$\{y > r \geq 0 \wedge x = r + qy\}$$

Proof obligations

take $t \triangleq r$

$$P \Rightarrow t \geq 0$$

$$(r \geq 0 \wedge y > 0 \wedge x = r + qy) \Rightarrow r \geq 0$$

$$\{P \wedge b \wedge t = z\} \text{ c } \{t < z\}$$

$$\{r \geq y > 0 \wedge \dots \wedge r = z\} \Rightarrow \{r \geq 0 \wedge y > 0 \wedge \dots \wedge r - y < z\}$$

$r := r - y;$

$$\{r \geq 0 \wedge y > 0 \wedge \dots \wedge r < z\}$$

$q := q + 1$

$$\{r \geq 0 \wedge y > 0 \wedge \dots \wedge r < z\} \Rightarrow \{r < z\}$$

Finding invariants is difficult!

{true}

```
 $k := 1;$   
 $r := x;$   
while  $k > 0$  do  
  if  $r > 100$  then  
     $r := r - 10;$   
     $k := k - 1$   
  else  
     $r := r + 11;$   
     $k := k + 1$ 
```

{ $r = f(x)$ }

what is
 f ?

McCarthy's 91 function

{true}

```
k := 1;  
r := x;  
while k > 0 do  
  if r > 100 then  
    r := r - 10;  
    k := k - 1  
  else  
    r := r + 11;  
    k := k + 1
```

{*r* = *f*(*x*)}

$$f(x) \triangleq \begin{cases} f(f(x + 11)) & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$$f(x) \triangleq \begin{cases} 91 & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

Invariant for McCarthy's 91 function?

{true}

$k := 1;$

$r := x;$

which
invariant?

{ ? }

while $k > 0$ do

 if $r > 100$ then

$r := r - 10;$

$k := k - 1$

 else

$r := r + 11;$

$k := k + 1$

{ ? } $\Rightarrow$ { $r = f(x)$ }

$$f(x) \triangleq \begin{cases} f(f(x + 11)) & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$$f(x) \triangleq \begin{cases} 91 & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

can you help me finding an invariant for the program:
"r:=x;q:=0;while (y<=r) do { r:=r-y; q:=q+1 }" ?

Yes! To find an invariant for the given program:

```
plaintext
r := x;
q := 0;
while (y <= r) do {
  r := r - y;
  q := q + 1;
}
```

we need to determine a property that holds **before and after each iteration** of the loop.

Step 1: Understanding the Program's Behavior

- The program initializes $r = x$ and $q = 0$.
- In each iteration, it subtracts y from r and increments q .
- The loop continues until $r < y$.
- Essentially, this computes the **quotient** q and the **remainder** r of the division $x \div y$, meaning:

$$x = q \cdot y + r$$

at the end of the program, where $0 \leq r < y$.

Step 2: Finding the Invariant

We want an expression that remains true throughout execution. Observing the program's logic, we see that **before and after each loop iteration**, the equation:

$$x = q \cdot y + r$$

remains valid. This is because:

- Initially: $q = 0$ and $r = x$, so $x = 0 \cdot y + x$ holds.
- In each iteration:
 - r is reduced by y , and q is increased by 1.
 - The equation transforms as:

$$x = (q + 1) \cdot y + (r - y)$$

which maintains the form $x = q \cdot y + r$.

Step 3: Expressing the Invariant Formally

Thus, the **loop invariant** is:

$$x = q \cdot y + r$$

which holds before entering the loop and remains true after each iteration.

Step 4: Verifying Initialization and Termination

- Initialization:** Before the loop starts, $q = 0$, $r = x$, so $x = 0 \cdot y + x$ is true.
- Preservation:** Each iteration updates q and r in a way that maintains the equation.
- Termination:** The loop stops when $r < y$. At this point, the final values of q and r satisfy:

$$x = q \cdot y + r, \quad 0 \leq r < y$$

which correctly represents the division.

Conclusion

The invariant for this program is:

$$x = q \cdot y + r$$

This property helps us reason about the correctness of the algorithm, which effectively computes the quotient and remainder of x divided by y .

Ask an expert?

can you help me finding an invariant for the program:
"r:=x;q:=0;while (y<=r) do { r:=r-y; q:=q+1 }" ?

The invariant for this program is:

$$x = q \cdot y + r$$

can you help me finding an invariant for the program:
"k:=1;r:=x;while (k>0) do { if (r>100) then { r:=r-10;k:=k-1} else {r:=r+11;k:=k+1} }" ?

The **loop invariant** for this program is:

$$r = x + 11k$$

can you help me finding an invariant for the program:
"k:=1;r:=x;while (k>0) do { if (r>100) then { r:=r-10;k:=k-1} else {r:=r+11;k:=k+1} }" ?

Yes! Let's analyze and find an invariant for the given program:

```
plaintext
k := 1;
r := x;
while (k > 0) do {
  if (r > 100) then {
    r := r - 10;
    k := k - 1;
  } else {
    r := r + 11;
    k := k + 1;
  }
}
```

Step 1: Understanding the Program's Behavior

- The program initializes $k = 1$ and $r = x$.
- The loop condition is $k > 0$, meaning it runs as long as k remains positive.
- The loop has two cases:
 - If $r > 100$:
 - r decreases by 10.
 - k decreases by 1.
 - If $r \leq 100$:
 - r increases by 11.
 - k increases by 1.
- The loop ends when k eventually reaches 0.

Step 2: Observing Changes to r and k

- Each iteration **modifies both k and r** :
 - If $r > 100$, we **decrease r and decrease k** .
 - If $r \leq 100$, we **increase r and increase k** .
- The key observation is that the program **fluctuates around 100**:
 - If r starts above 100, it tends to decrease.
 - If r starts below or at 100, it tends to increase.
- The interplay between increasing and decreasing k ensures termination.

Step 3: Finding the Invariant

We need to find a property that remains true **before and after every iteration**.

Observing the changes:

- Each time k changes, r is updated by a constant amount.
- Specifically:
 - If k increases by 1, r increases by 11.
 - If k decreases by 1, r decreases by 10.

This suggests the key invariant:

$$r = x + 11k$$

which we will **prove by induction**.

Step 4: Verifying the Invariant

Base Case (Before the Loop Starts)

- Initially: $k = 1, r = x$.
- Plugging into the invariant: $r = x + 11(1) = x + 11$

This holds because $r = x$.

Inductive Step (Loop Iteration)

- Suppose the invariant holds before an iteration, i.e., $r = x + 11k$.
- Now, consider both cases:
 - If $r > 100$:
 - r changes: $r' = r - 10$.
 - k changes: $k' = k - 1$.
 - New equation: $r' = x + 11k - 10 = x + 11(k - 1) + 1$ which still follows the form $r' = x + 11k'$.
 - If $r \leq 100$:
 - r changes: $r' = r + 11$.
 - k changes: $k' = k + 1$.
 - New equation: $r' = x + 11k + 11 = x + 11(k + 1)$ which still follows $r' = x + 11k'$.

Thus, the invariant **remains true** after every iteration.

Step 5: Confirming the Invariant at Termination

- The loop stops when $k = 0$.
- When $k = 0$, the invariant says:

$$r = x + 11(0) = x$$

So the final value of r is x , showing correctness.

Conclusion

The **loop invariant** for this program is:

$$r = x + 11k$$

This equation remains true throughout execution, ensuring that the changes in k and r are always balanced according to their relationship.

Invariant for McCarthy's 91 function?

{true}

$k := 1;$

$r := x;$



don't
think so!

when $(k = 1, r = x)$ then $r = x + 11k$ does not hold!

{ $r = x + 11k$ }

while $k > 0$ do

 if $r > 100$ then

$r := r - 10;$

$k := k - 1$

 else

$r := r + 11;$

$k := k + 1$

{ ? } $\Rightarrow$ { $r = f(x)$ }

$$f(x) \triangleq \begin{cases} f(f(x + 11)) & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$$f(x) \triangleq \begin{cases} 91 & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

Invariant for McCarthy's 91 function

{true}

$k := 1;$

$r := x;$ when $(k = 1, r = x)$ then both $k \geq 0$ and $f^k(r) = f^1(x) = f(x)$ hold

$\{k \geq 0 \wedge f^k(r) = f(x)\}$

while $k > 0$ do

 if $r > 100$ then

$r := r - 10;$

$k := k - 1$

 else

$r := r + 11;$

$k := k + 1$

$$f(x) \triangleq \begin{cases} f(f(x + 11)) & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$$f(x) \triangleq \begin{cases} 91 & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$\{k = 0 \wedge f^k(r) = f(x)\} \Rightarrow \{r = f(x)\}$

Variant for McCarthy's 91 function?

{true}

$k := 1;$

$r := x;$

$\{k \geq 0 \wedge f^k(r) = f(x)\}$

$t \triangleq ?$

which variant
(for termination)?

while $k > 0$ do

if $r > 100$ then

$r := r - 10;$

$k := k - 1$

else

$r := r + 11;$

$k := k + 1$

$\{k = 0 \wedge f^k(r) = f(x)\} \Rightarrow \{r = f(x)\}$

$$f(x) \triangleq \begin{cases} f(f(x + 11)) & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$$f(x) \triangleq \begin{cases} 91 & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

Finding invariants (McCarthy91)

{true}

$k := 1;$

$r := x;$

$\{k \geq 0 \wedge f^k(r) = f(x)\}$

$t = (|101 - r + 10k|, k)$

lexicographic
order

while $k > 0$ do

if $r > 100$ then

$r := r - 10;$

$k := k - 1$

else

$r := r + 11;$

$k := k + 1$

$\{k = 0 \wedge f^k(r) = f(x)\} \Rightarrow \{r = f(x)\}$

$$f(x) \triangleq \begin{cases} f(f(x + 11)) & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

$$f(x) \triangleq \begin{cases} 91 & x \leq 100 \\ x - 10 & \text{otherwise} \end{cases}$$

Correctness

Th. Any derivable HL triple is valid

Proof. By induction on the derivation tree.

Remember the proof for composition of commands

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}}$$

$$\llbracket c_1; c_2 \rrbracket P = \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket P) \subseteq \llbracket c_2 \rrbracket R \subseteq Q$$

Floyd's axiom soundness

$$\{P\} x := a \{ \exists x'. P[x'/x] \wedge x = a[x'/x] \}$$

Let us take $\delta \in \llbracket x := a \rrbracket P$,

we need to show that $\delta \models \exists x'. P[x'/x] \wedge x = a[x'/x]$

It must be the case that $\delta = \sigma[v/x]$ for some $\sigma \models P$ such that $v = \llbracket a \rrbracket \sigma$

Let $x' = \sigma(x)$. Then $\delta(x) = v$ and

$$\llbracket a[x'/x] \rrbracket \delta = \llbracket a[x'/x] \rrbracket \sigma[v/x] = \llbracket a[x'/x] \rrbracket \sigma = \llbracket a \rrbracket \sigma = v.$$

Therefore, $\delta \models (x = a[x'/x])$.

Moreover, $\llbracket P[x'/x] \rrbracket \delta = \llbracket P[x'/x] \rrbracket \sigma[v/x] = \llbracket P[x'/x] \rrbracket \sigma = \llbracket P \rrbracket \sigma = \text{true}$

Therefore, $\delta \models P[x'/x]$.

From which we conclude $\delta \models \exists x'. P[x'/x] \wedge x = a[x'/x]$

Conditional rule: correctness

$$\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}$$

$$\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}$$

We assume the validity of the premises:

$$\llbracket c_1 \rrbracket(P \wedge b) \subseteq Q \text{ and } \llbracket c_2 \rrbracket(P \wedge \neg b) \subseteq Q$$

Therefore:

$$\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket P = \llbracket b?; c_1 + \neg b?; c_2 \rrbracket P$$

$$= \llbracket c_1 \rrbracket(\llbracket b \rrbracket P) \cup \llbracket c_2 \rrbracket(\llbracket \neg b \rrbracket P)$$

$$= \llbracket c_1 \rrbracket(P \wedge b) \cup \llbracket c_2 \rrbracket(P \wedge \neg b) \subseteq Q \cup Q = Q$$

While rule: correctness

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}}$$

We assume the validity of the premise: $\llbracket c \rrbracket (P \wedge b) \subseteq P$, i.e., $\llbracket b?; c \rrbracket P \subseteq P$

It can be proved by induction that, for any $k \in \mathbb{N}$ it holds $\llbracket b?; c \rrbracket^k P \subseteq P$

Therefore:

$$\llbracket \text{while } b \text{ do } c \rrbracket P = \llbracket (b?; c)^*; \neg b? \rrbracket P = \llbracket \neg b \rrbracket \left(\bigcup_{k=0}^{\infty} \llbracket b?; c \rrbracket^k P \right)$$

$$\subseteq \llbracket \neg b \rrbracket \left(\bigcup_{k=0}^{\infty} P \right) = \llbracket \neg b \rrbracket P = (P \wedge \neg b)$$

Incompleteness I

Conjecture Any valid HL triple is derivable

Counterexample:

$\{\text{true}\} \ c \ \{\text{false}\}$ is valid only when c diverges
but halting problem is not r.e.
while the set of derivable HL triples is r.e.

Incompleteness II

Conjecture Any valid HL triple is derivable

Counterexample:

$\{\text{true}\}$ skip $\{Q\}$ is valid when Q is a tautology

but Godel's Incompleteness Theorem (1939) tells us that there is no *effective* proof system such that its theorems coincide with all valid arithmetic assertions

Relative completeness I

Relative completeness: suppose we can consult an oracle to check if an assertion $P \Rightarrow P'$ is valid or not, then HL is complete

In other words, we separate concerns about programs and reasoning about them from concerns to do with arithmetic and the incompleteness of any proof system for it

Dijkstra's weakest precondition

Given a command c and a postcondition Q a **weakest liberal precondition** is a predicate P such that for any precondition R

$$\{R\} c \{Q\} \text{ iff } R \Rightarrow P$$

i.e., P is the least restrictive requirement that guarantees that Q holds after executing c (if it terminates)

Typically, it is denoted by $wlp(c, Q) \triangleq \{\sigma \in \Sigma \mid [[c]]\{\sigma\} \subseteq Q\}$

Adjoints

$$P \Rightarrow wlp(c, Q)$$

iff

$$[[c]]P \subseteq Q$$

(Relative) Completeness

for any postcondition Q expressible in the logic
and for any command c , the precondition
 $wlp(c, Q)$ is also expressible in the logic

Th. If the logic language is *expressive enough*, then any valid HL triple can be derived.

Proof. Suppose $\{P\} c \{Q\}$ is valid (with P and Q expressible).
By structural induction on c we can build an assertion R that is
equivalent to $wlp(c, Q)$ and such that $\{R\} c \{Q\}$ is derivable.
By applying the consequence rule we derive $\{P\} c \{Q\}$.

Adding nondeterminism

Regular commands

regular
command

atomic
command

$r ::= e$

$r_1; r_2$

$r_1 + r_2$

$r^\star$

choice

Kleene
star

$e ::= \text{skip} \mid x := a \mid b? \mid \dots$

$\llbracket b? \rrbracket P \triangleq \llbracket b \rrbracket P$

$\llbracket r_1 + r_2 \rrbracket P \triangleq \llbracket r_1 \rrbracket P \cup \llbracket r_2 \rrbracket P$

$\llbracket r^\star \rrbracket P \triangleq \bigcup_{k=0}^{\infty} \llbracket r \rrbracket^k P$

Encoding while commands

if b then c_1 else $c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$

while b do $c \triangleq (b?; c)^{\star}; \neg b?$

Minimal set of rules

$$\frac{}{\{P\} e \{\llbracket e \rrbracket P\}} \{\text{atom}\} \qquad \frac{\{P\} r_1 \{R\} \quad \{R\} r_2 \{Q\}}{\{P\} r_1; r_2 \{Q\}} \{\text{seq}\}$$

$$\frac{\forall i \in \{1,2\} \quad \{P\} r_i \{Q\}}{\{P\} r_1 + r_2 \{Q\}} \{\text{choice}\} \qquad \frac{\{P\} r \{P\}}{\{P\} r^\star \{P\}} \{\text{iter}\}$$

$$\frac{P \Rightarrow P' \quad \{P'\} r \{Q'\} \quad Q' \Rightarrow Q}{\{P\} r \{Q\}} \{\text{cons}\}$$

Auxiliary rules

$$\frac{\{P_1\} \, r \, \{Q_1\} \quad \{P_2\} \, r \, \{Q_2\}}{\{P_1 \vee P_2\} \, r \, \{Q_1 \vee Q_2\}} \text{disj}$$

assigned variables in r
are disjoint from
free variables in R

$$\frac{\{P_1\} \, r \, \{Q_1\} \quad \{P_2\} \, r \, \{Q_2\}}{\{P_1 \wedge P_2\} \, r \, \{Q_1 \wedge Q_2\}} \text{conj}$$

$$\frac{\{P\} \, r \, \{Q\}}{\{P \wedge R\} \, r \, \{Q \wedge R\}} \text{frame}$$

$$\frac{P \Rightarrow P' \quad \{P'\} \, r \, \{Q\}}{\{P\} \, r \, \{Q\}} \text{stren}$$

$$\frac{\{P\} \, r \, \{Q'\} \quad Q' \Rightarrow Q}{\{P\} \, r \, \{Q\}} \text{weak}$$

Question 1

Can we take $P = \neg b$ an invariant?

$$\frac{\{P \wedge b\} \ c \ \{P\}}{\{P\} \ \text{while } b \ \text{do } c \ \{P \wedge \neg b\}}$$

$$\frac{\{\text{false}\} \ c \ \{\neg b\}}{\{\neg b\} \ \text{while } b \ \text{do } c \ \{\neg b\}} \quad \checkmark$$

Question 2

Find a derivation for the HL triple

$\{\text{true}\}$ if $x \geq y$ then $z := x$ else $z := y$ $\{z = \max(x, y)\}$

Question 3

Prove that rule {conj} is sound

$$\frac{\{P_1\} \textit{r} \{Q_1\} \quad \{P_2\} \textit{r} \{Q_2\}}{\{P_1 \wedge P_2\} \textit{r} \{Q_1 \wedge Q_2\}} \textit{conj}$$

Question 4

Show that the following rule for assignment is not sound

$$\frac{}{\{P\} x := a \{P[a/x]\}}$$

syntax
replacement